# TripSync

**Wspólne wyjazdy. Plan, zadania i rozliczenia - także offline.**




## Cel i zakres

Aplikacja dla niewielkich grup znajomych organizujących wspólne wyjazdy. Łączy harmonogram, zadania, listę pakowania, propozycje atrakcji i rozliczanie kosztów. Dane są dostępne bez internetu, a po odzyskaniu połączenia synchronizowane z własnym backendem REST.

Zakres dla trzech osób rozszerza podstawowe wymagania o współpracę wielu kont, role i kontrolę dostępu, wykrywanie konfliktów edycji, kolejkę operacji oraz dokładny algorytm rozliczeń.

| Platforma | Dostarczany rezultat |
| --- | --- |
| Android 8.0+ / Kotlin | Kod źródłowy, APK debug, backend SQLite/Python |
| MVVM / Jetpack Compose / Material 3 | Ekrany, testy, diagramy, instrukcja i screeny |
| Działanie lokalne i online | Demo bez konta; współdzielenie po uruchomieniu API |

## Scenariusz użytkownika

Organizator tworzy wyjazd i udostępnia kod zaproszenia. Grupa ustala plan, głosuje na pomysły i dzieli zadania. Każdy zapisuje poniesione koszty, także offline. Po synchronizacji wszyscy widzą aktualne dane, a ekran kosztów pokazuje salda i propozycje przelewów.

## Granice projektu

Jedna waluta na wyjazd: PLN, EUR albo USD. Bez czatu, rezerwacji, realizowania płatności i pełnych map offline. Synchronizacja jest okresowa lub ręczna; aplikacja nie deklaruje aktualizacji w czasie rzeczywistym.

# Wymagania

## Funkcjonalne

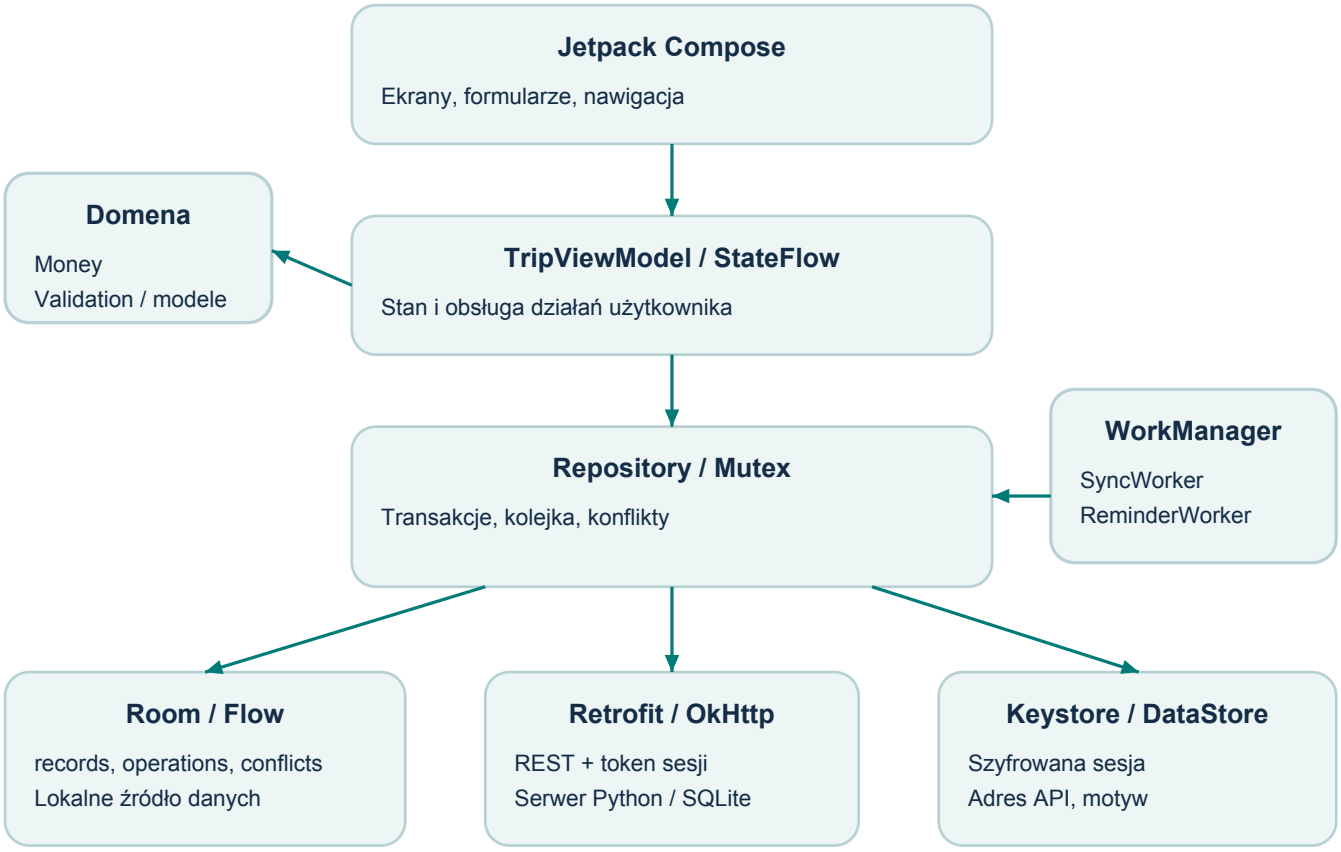
| ID | Wymaganie |
|---|---|
| F01 | Konta: rejestracja, logowanie, szyfrowana sesja i wylogowanie. |
| F02 | CRUD wyjazdów: nazwa, miejsce, daty, opis i waluta; rola organizatora. |
| F03 | Uczestnicy: dołączenie kodem/linkiem, rotacja kodu, usuwanie dostępu. |
| F04 | Harmonogram: CRUD, godzina, miejsce, kolejność i wykonanie. |
| F05 | Propozycje: CRUD, jeden własny głos, cofnięcie i ponowne oddanie głosu. |
| F06 | Przeniesienie propozycji do harmonogramu przez formularz. |
| F07 | Zadania i pakowanie: CRUD, przypisanie osoby, oznaczanie wykonania. |
| F08 | Wydatki: CRUD, płatnik, data, uczestnicy i dwa sposoby podziału kosztu. |
| F09 | Salda i propozycje przelewów z dokładnością grosza/centa. |
| F10 | Trwałość danych i lokalny CRUD bez internetu. |
| F11 | Synchronizacja, kolejka operacji, ponowienia i konflikty wersji. |
| F12 | Lokalne przypomnienia o punktach planu i zadaniach. |
| F13 | Wieloekranowa nawigacja, zachowanie formularza, ustawienia motywu. |
| F14 | Walidacja i czytelne komunikaty błędów sieci, bazy oraz serwera. |
| F15 | Edytowalne demo bez konta i systemowe udostępnianie planu. |

## Niefunkcjonalne

**N01-N03:** Material Design 3, responsywne widoki, modularny kod i asynchroniczne operacje Room/Retrofit poza głównym wątkiem. **N04:** odporność na odtworzenie aktywności przez ViewModel, rememberSaveable i Room. **N05-N06:** kontrola dostępu, Keystore, HTTPS, dostępność i etykiety kontrolek. **N07-N09:** testy domeny, UI i integracji, możliwość rozbudowy, historia Git i komplet materiałów do uruchomienia.

# Architektura aplikacji

## MVVM z wydzieloną domeną



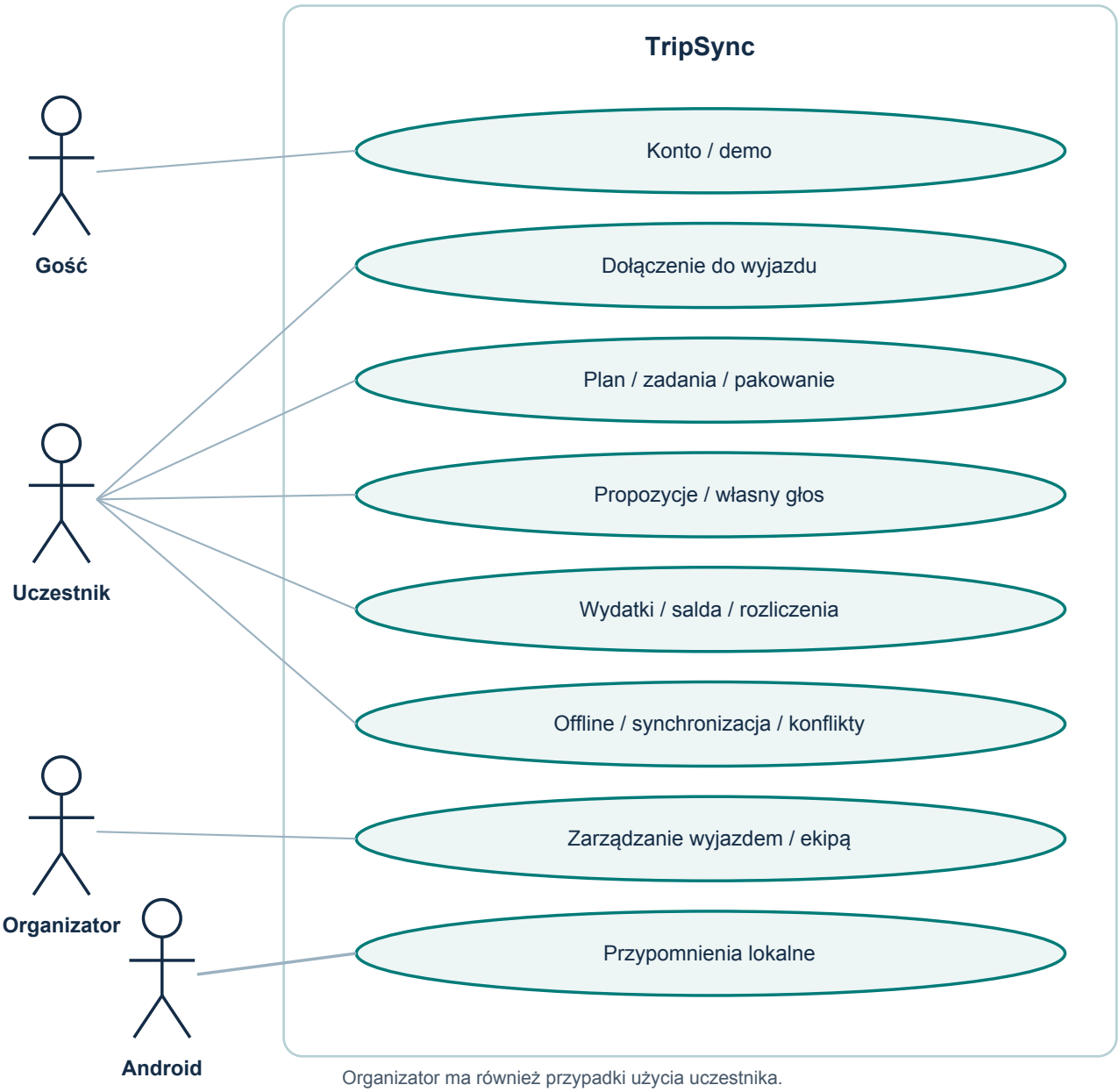

UI odczytuje Room. Sieć aktualizuje lokalną bazę; nie blokuje ekranu.

## Przepływ i synchronizacja

1. Formularz waliduje dane, a repozytorium zapisuje rekord i operację w jednej transakcji Room.
2. Ekrany otrzymują dane z lokalnej bazy przez Flow i StateFlow.
3. SyncWorker wysyła operacje, a serwer sprawdza sesję, rolę, dane i bazową wersję.
4. Identyfikator operacji zapobiega podwójnemu zapisowi po ponowieniu.
5. Różnica wersji tworzy konflikt. Użytkownik wybiera lokalną albo zdalną wersję.
6. Snapshot uzupełnia Room, zachowując niewysłane lokalne zmiany. Utrata dostępu usuwa lokalny wyjazd po udanej synchronizacji.

Usunięcia są przesyłane jako tombstone. Mutex porządkuje zapisy i synchronizację. Dane z aktywnym konfliktem czekają na rozstrzygnięcie; nie są automatycznie nadpisywane.

# Przypadki użycia




Organizator ma również przypadki użycia uczestnika.

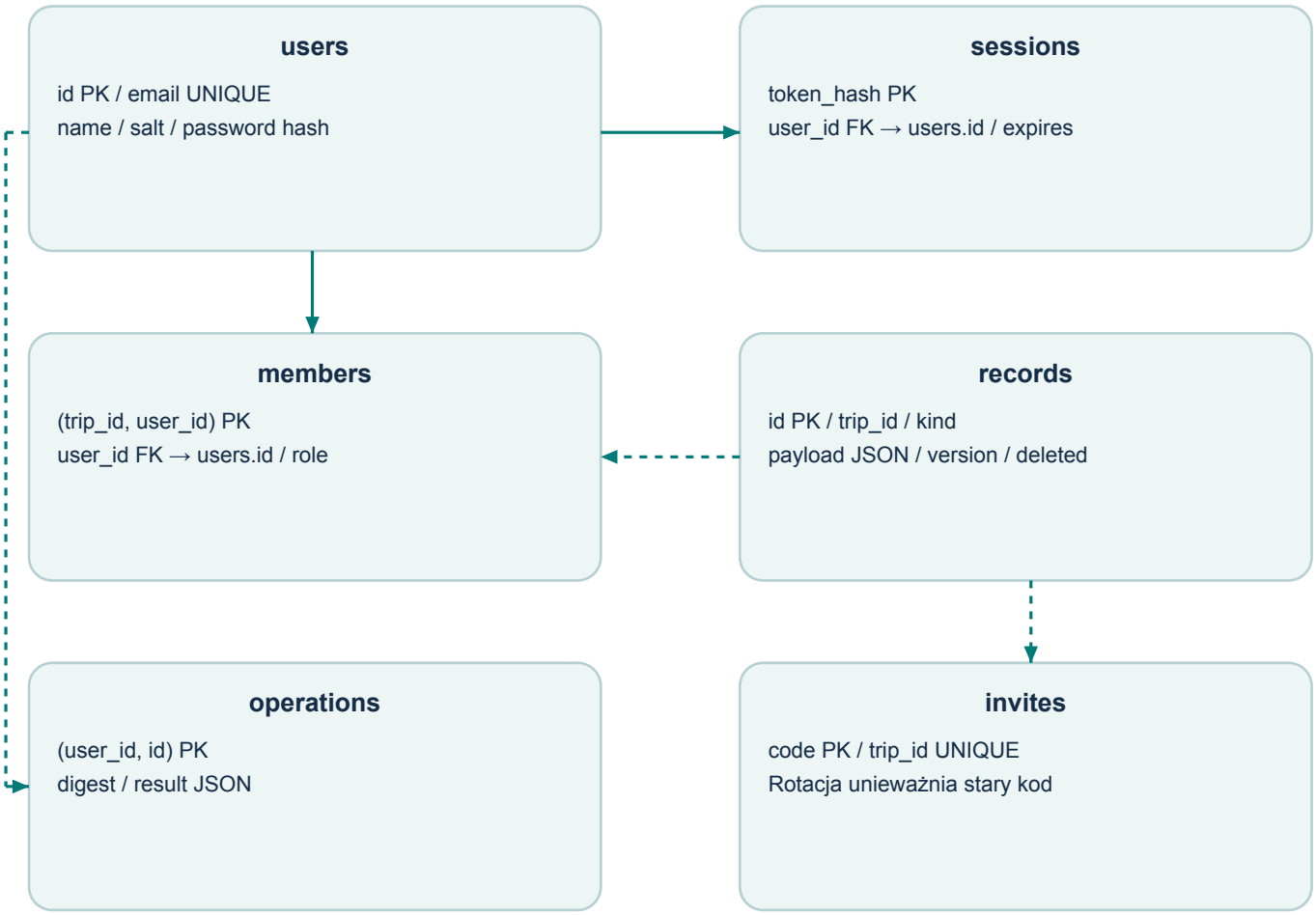
## Role i uprawnienia

**Gość:** konto albo lokalne demo. **Uczestnik:** wspólny plan, zadania, koszty i propozycje; głos wyłącznie za siebie. **Organizator:** dodatkowo tworzenie, edycja i usuwanie wyjazdu oraz zaproszenia i zarządzanie uczestnikami.

**System Android:** WorkManager uruchamia synchronizację i lokalne przypomnienia. Organizator nie może usunąć siebie ani uczestnika mającego aktywne wydatki; wcześniej należy zmienić te koszty.

# Diagramy baz danych

## Backend: fizyczne tabele SQLite




Strzałki ciągłe: fizyczne powiązania FK do users. Linie przerywane: powiązania logiczne obsługiwane w API. Rekordy podrzędne wskazują wyjazd przez trip_id; payload JSON zawiera pola danego typu.

## Urządzenie: Room




Relacje lokalne są logiczne, bez fizycznych FK. Unikalny indeks operations.recordId zapewnia jedną oczekującą operację na rekord. Indeksy records.tripId i records.kind wspierają odczyty. Spójność zapisów zapewniają transakcje repozytorium. Schemat Room jest dołączony w app/schemas.

# Technologie i uruchomienie

| Technologia | Zastosowanie |
| --- | --- |
| Kotlin / AGP 9.1.1 / SDK 36.1 | Android; Gradle Wrapper 9.3.1, JDK 17 lub 21 |
| Compose BOM 2024.01.00 / Material 3 | Interfejs, motyw, kontrolki i dostępność |
| Navigation Compose 2.7.7 / Lifecycle 2.9.0 | Nawigacja i ViewModel; zależności rozwiązywane przez Gradle |
| Room 2.8.4 / KSP 2.2.10-2.0.2 | Lokalna baza i generowany DAO |
| Coroutines 1.10.2 / Flow | Asynchroniczny odczyt i reaktywny stan |
| Retrofit 2.9.0 / OkHttp 4.12.0 / Gson 2.11.0 | REST oraz kodowanie JSON |
| DataStore 1.1.7 / WorkManager 2.9.0 | Ustawienia, synchronizacja i przypomnienia |
| Keystore / AES-GCM | Szyfrowane przechowywanie sesji |
| Python 3.10+ / SQLite / unittest | Backend bez dodatkowych pakietów |
| JUnit 4 / Compose UI Test / AndroidJUnitRunner | Testy jednostkowe i urządzeniowe |

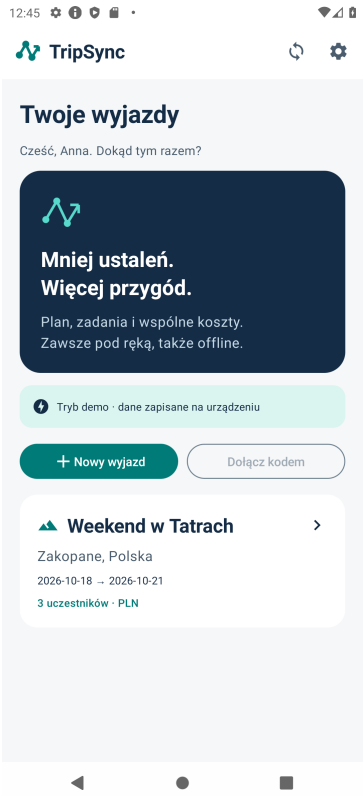
## Instalacja i start

Najprościej: zainstaluj **output/apk/TripSync-debug.apk** i wybierz **Wypróbuj bez konta**. Do współpracy uruchom serwer, ustaw adres API przed zalogowaniem i utwórz oddzielne konta.

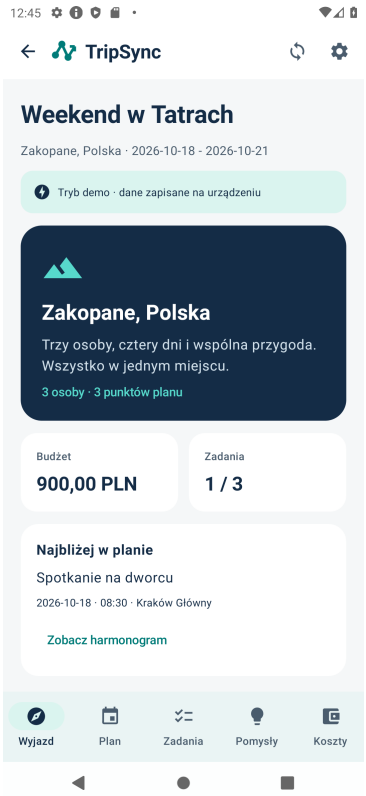
```
python3 backend/server.py
./gradlew :app:assembleDebug
./gradlew :app:testDebugUnitTest
```

Emulator debug: http://10.0.2.2:8080/. Telefon przez USB: adb reverse tcp:8080 tcp:8080 i adres http://127.0.0.1:8080/. Pozostałe urządzenia wymagają API HTTPS z zaufanym certyfikatem. Otwórz katalog w Android Studio i ustaw własne sdk.dir w local.properties. Pełne komendy, uruchomienie TLS i testy są w README.md.

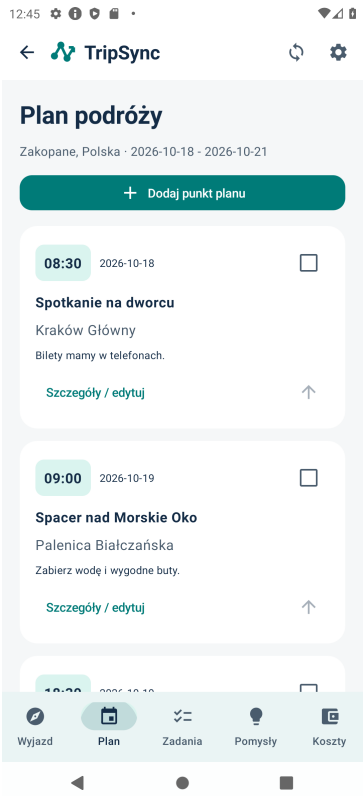
# Najważniejsze widoki / 1



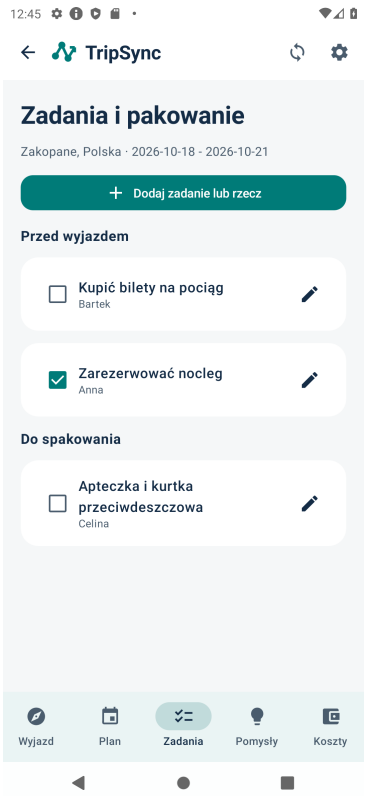

Lista wyjazdów i status danych




Podsumowanie wyjazdu




Harmonogram i wykonanie

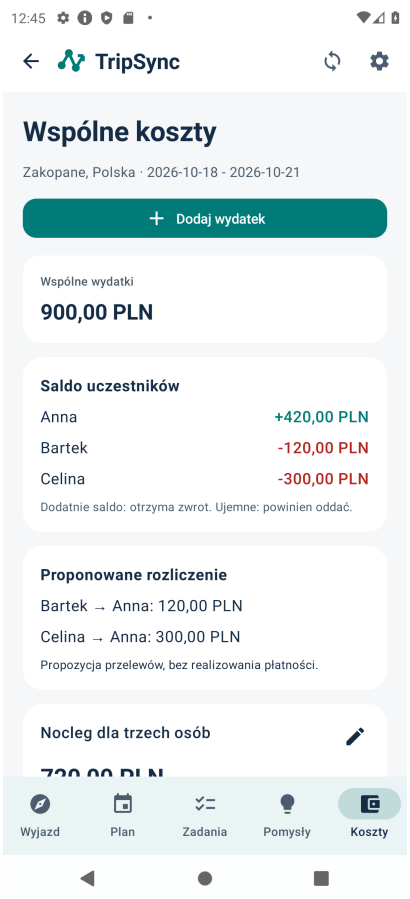



Zadania oraz pakowanie

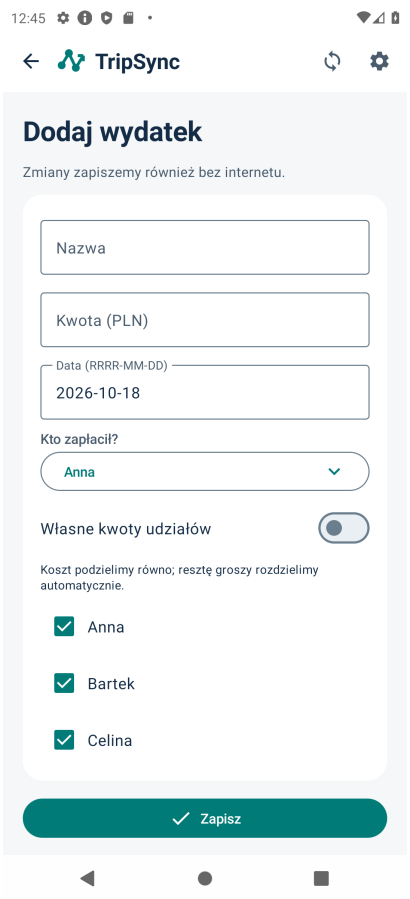
# Najważniejsze widoki / 2




Propozycje i głosowanie




Wydatki, salda i przelewy




Formularz wydatku

## Źródło zrzutów

Obrazy pochodzą z działającej aplikacji na emulatorze Android, a nie z makiet. Test UI przechodzi przez główne zakładki i zapisuje rzeczywiste screeny. Dane przykładowe: trzy osoby, nocleg 720 PLN i bilety 180 PLN.

## Interpretacja rozliczeń

Przy równym podziale kosztów Anna ma saldo +420 PLN, Bartek -120 PLN, a Celina -300 PLN. Propozycja rozliczenia: Bartek oddaje Annie 120 PLN, Celina oddaje Annie 300 PLN. Suma sald wynosi zero.

## Nawigacja

Poza pokazanymi widokami dostępne są logowanie/rejestracja, uczestnicy z zaproszeniami, formularze pozostałych danych, ustawienia oraz rozstrzyganie konfliktów.

# Testy i rezultaty

## Weryfikacja wykonana 04.10.2026

| Obszar | Wynik |
|---|---|
| Backend Python | 14 testów / 0 niepowodzeń |
| Domena Kotlin / JUnit | 11 testów / 0 niepowodzeń |
| Android: interfejs i REST | 11 testów / 0 niepowodzeń |
| Budowanie APK | assembleDebug: zakończone powodzeniem |
| Android Lint | 0 błędów / 1 ostrzeżenie |

## Zakres sprawdzeń

Domena: dokładne kwoty, reszty groszowe, błędne udziały, walidacja dat i godzin oraz 200 losowych scenariuszy rozliczeń. Backend: hashowanie, sesje, role, zaproszenia, idempotencja, konflikty, tombstone, głosowanie, wydatki i trwałość bazy.

Android: CRUD wydatku i zmiana salda, błędy formularza, zachowanie formularza po odtworzeniu aktywności, ponowne głosowanie, odtworzenie danych i główne widoki. Integracja REST: dwa oddzielne konta/repozytoria, lokalna kolejka, równoczesna edycja, wybór obu sposobów rozwiązania konfliktu, usunięcia i cofnięcie dostępu.

## Środowisko i dowody

Emulator Medium_Phone, Android 10 / API 29, ARM64, 1080 × 2400. Kompilacja: SDK 36.1, JDK 21 i Gradle 9.3.1. Testy backendu wykonano na Pythonie 3.13. Raporty XML, raport Lint i logi znajdują się w output/reports.

## Ograniczenia weryfikacji

Nie wykonano testów na fizycznym telefonie, audytu bezpieczeństwa ani testów obciążeniowych. Interwały powiadomień i pracy w tle podlegają ograniczeniom Androida. Końcowy Lint uruchomiono offline: 1 ostrzeżenie wersji Lifecycle. Sprawdzenia dostępności nowszych wersji przez sieć nie należą do tego przebiegu.

## Bezpieczeństwo i zakres odpowiedzialności

Sesja AES-GCM/Keystore; prywatna baza lokalna bez SQLCipher; wyłączone backup i transfer danych. Serwer: PBKDF2-SHA256, losowe tokeny ważne 7 dni, kontrola dostępu i limiter logowania. Backend jest dydaktyczny i wymaga własnego hostingu HTTPS do pracy poza lokalnym środowiskiem.

# Organizacja i oddanie

## Podział folderów

| Folder | Odpowiedzialność |
| --- | --- |
| backend/ | API, konta, uprawnienia, synchronizacja i testy |
| app/.../domain/ | Modele, walidacja i rozliczenia |
| app/.../data/ | Room, API, sesja, ustawienia i kolejka |
| app/.../ui/screens/ | Oddzielne foldery funkcji: auth, trips, trip, plan, tasks, ideas, money, editor, members, conflicts, settings |
| app/src/test/ oraz androidTest/ | Testy Kotlin, Compose i rzeczywistego REST |
| docs/ oraz scripts/ | Dokumentacja, Mermaid, screeny i odtwarzalne budowanie |
| output/ | APK, PDF, raporty oraz paczka projektu |

## Proponowany podział odpowiedzialności

**Osoba 1:** backend, konta i wyjazdy. **Osoba 2:** interfejs, plan, zadania i propozycje. **Osoba 3:** koszty, algorytm i synchronizacja. Każdy obszar obejmuje testy i dokumentację. Podział umożliwia niezależne rozwijanie modułów i wspólną integrację.

## Repozytorium Git

Projekt ma lokalne repozytorium i rzeczywistą historię etapów pracy. Nie wskazano zdalnego repozytorium, dlatego dokumentacja nie zawiera fikcyjnego publicznego linku. Po publikacji należy dopisać adres repozytorium. Dołączony Git bundle zachowuje historię.

## Materiały do przekazania

Źródła Android i backendu, Gradle Wrapper, APK debug, dokumentacja PDF/Markdown, diagramy, schemat Room, screeny i raporty testów. Paczka pomija lokalne ścieżki SDK, cache, tokeny i bazy testowe.

## Rozbudowa

Możliwe kolejne kroki: większe testy urządzeniowe, paginacja synchronizacji, hosting HTTPS, odzyskiwanie hasła i weryfikacja adresu e-mail. Funkcje te nie są przedstawiane jako już zaimplementowane.